Python Multiprocessing Cheat Sheet

Overview

The multiprocessing module in Python allows concurrent execution of code by creating separate processes, enabling true parallelism on multi-core systems. It bypasses the Global Interpreter Lock (GIL), unlike threading.

Core Concepts

- Process: Independent instance of the Python interpreter executing code.
- Pool: Manages a pool of worker processes for parallel execution.
- Queue: Safe data exchange between processes.
- Pipe: Enables two-way communication between processes.
- Lock/Semaphore: Synchronization primitives to prevent data races.
- Manager: Allows shared state across processes.

Commonly Used Classes & Functions

- `multiprocessing.Process(target, args=())`: Creates and runs a new process.
- `multiprocessing.Pool(processes)`: Creates a pool of worker processes.
- `multiprocessing.Queue()`: FIFO queue for inter-process communication.
- `multiprocessing.Pipe()`: Returns a pair of connection objects for communication.
- `multiprocessing.Lock()`: Prevents concurrent access to shared resources.
- `multiprocessing.Manager()`: Provides shared objects like lists and dicts.

Example Usage

```
from multiprocessing import Process
import os

def worker(name):
    print(f'Process {name} running in PID {os.getpid()}')

if __name__ == '__main__':
    p1 = Process(target=worker, args=('A',))
    p2 = Process(target=worker, args=('B',))
    p1.start()
    p2.start()
    p1.join()
    p2.join()
```

```
from multiprocessing import Pool

def square(x):
    return x * x

if __name__ == '__main__':
    with Pool(4) as pool:
        results = pool.map(square, [1, 2, 3, 4])
        print(results)
```

Tips and Best Practices

- Always protect the entry point with `` `if __name__ == '__main__':` `` on Windows.
    - Good practice to use this on all operating systems
- Avoid sharing large data structures — use Queues or Managers for safe sharing.
- Use Pool for batch parallel processing tasks.
- Handle process termination gracefully using join() and terminate().
- Profile and measure performance — multiprocessing adds overhead.

Common Pitfalls

- Forgetting `if __name__ == '__main__':` causes recursion errors on Windows.
- Not joining processes can lead to zombie processes.
- Shared state isn't automatically synchronized — use Lock or Manager.